production monitoring up to another team—if it's done at all. As you're building your features you should be thinking about your consumer, how they're using your product, and what you need to know to improve their experience. This mindset will help you do the right type of work to get good data that can help you determine if you're hitting your true goal: giving your consumer what they want.

Two example frameworks that help with this concept are StatsD (github.com/etsy/statsd/) and Atlas (github.com/Netflix/atlas/wiki). The concept behind them fits precisely with what we've been discussing so far and a mentality that shouldn't be much of a stretch: "measure anything, measure everything." StatsD and Atlas allow developers to easily add arbitrary metrics and telemetry in their code. These libraries are embedded into your code with the intention of pushing the ownership of arbitrary monitoring onto the development team.

These systems work by installing an agent into your application container or a daemon on your server that listens for metrics as they're being sent from your application. That component then sends the metrics back to a central time-data series server for indexing. This process is illustrated in figure 6.4.

Figure 6.4. An example architecture to collect arbitrary metrics